Chronicle

From graph memory to video-free memory

A combined architecture guide

Learn the whole system in one sitting — 27 October 2026

Combines the condensed graph specification with the VFM (VLM) architecture note

Why this document exists. Chronicle has two architectural layers that are usually explained separately: the **graph memory** (perception, object identity, 9-state lifecycle, GraphRAG) and the **video-free memory** (VLM captions, event ledger, packed vault, deterministic answers). They are not competing designs. They are two halves of one system. This guide explains each half, then shows exactly how they join.

Graph memory in one sentence	VFM memory in one sentence
Perception gates heavy models behind motion and contact, tracks objects through a 9-state lifecycle, and stores a small dual graph whose 2-hop neighbourhood answers questions with timestamped citations.	A video episode is compiled once into timestamped VLM captions, a compressed event ledger, a few WebP evidence stills and an LLM-ready pack, so questions are answered from text in microseconds with no video in the query path.

The combined thesis. *The graph is the index; the captions and events are the recallable memory; the vault is the fast answer engine; evidence makes it auditable.* Compile once, query many times, and never re-watch the video to answer a question.

Contents at a glance

Part	Sections
Foundations	1 How to use this guide · 2 The one-page mental model · 3 Two architectures compared
I. Graph memory	4 Four modules · 5 Data model and 9-state lifecycle · 6 Perception · 7 Storage · 8 Query
II. VFM layer	9 Why a second layer · 10 Compile pipeline · 11 Artefacts · 12 Query, latency, counts, evidence
III. Putting it together	13 Two ways in · 14 Routing a question · 15 Trace end to end · 16 Same UI, two engines
IV. Study aids	17 Numbers · 18 Industry comparison · 19 Limits · 20 Demo · 21 Self-check · 22 Glossary · 23 Commands · 24 Reading

1 How to use this guide

This is a study guide, not a specification. It is written to be read in three passes.

Pass	Read	Goal	Time
1	Sections 2–3	Get the one-page model and the division of labour between the two layers	10 min
2	Parts I–III	Understand the graph, the VFM layer, and how a question is routed	30 min
3	Part IV	Memorise the numbers, run the self-check, rehearse the demo	15 min

Two boxes appear throughout:

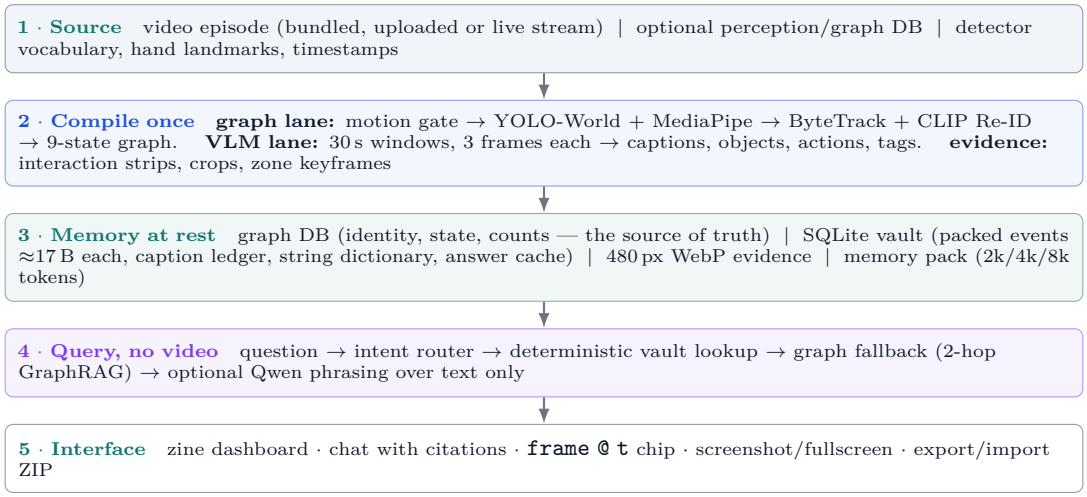
- **Key idea** boxes are the five or six things you should be able to explain without notes.
- **Checkpoint** boxes are quick recall questions. Answer them before moving on; the answer is given in italics so you can self-test.

By the end you should be able to: (i) draw the compile-and-query pipeline from memory, (ii) explain why the graph stays small, (iii) explain what VFM adds and what it loses, (iv) route a question to the right engine, and (v) defend the design against the obvious judge questions.

2 The one-page mental model

The problem both layers solve

Video is expensive to keep, expensive to search, and expensive to re-read. A 15-minute walk can be 249 MB; a question about second 10 should not require decoding the file, sampling frames and calling a vision model. Chronicle's answer is to pay the expensive cost *once*, at compile time, and then answer from compact structures.



Read the diagram top to bottom. Everything above layer 4 is paid once per episode. Everything in layer 4 is paid once per question. That split is the entire performance argument.

Key idea. Do not memorise features; memorise the split. Expensive perception and VLM work happens at compile time. Query time touches only small text structures. The video is kept for playback and screenshots, never for answering.

3 Two architectures side by side

	Graph memory (original)	VFM memory (VLM layer)
What is built	A per-episode SQLite graph of entities, states, edges, zones and anchors	A vault of timestamped captions, coalesced events, evidence references and an answer cache
When compute happens	Per frame, but gated by motion, geometry and state transitions	Once per episode, in the background, over 30 s caption windows
What it knows	Identity, support, containment, interaction, exact counts	Natural-language “what happened”, whole-episode coverage, appearance and activity
Answer engine	Entity resolution + 2-hop GraphRAG + templates	Intent lookup over the vault; graph fallback on a miss
Typical latency	0.14–1.05 ms	0.05–0.06 ms (demo); 0.89 ms cold on a 15-min walk
Video needed to answer	No	No
Main strength	Exact state, edges, zones, count reconciliation, audit trail	Semantic recall, temporal coverage, sub-millisecond deterministic answers
Main weakness	No semantic answer if the clip has no VLM interaction table	Lossy captions; the VLM can hallucinate detail
Role in the combined system	Index, source of truth, fallback, count authority	First answer engine and recall layer

Key idea. They are complementary by construction. The graph answers *what state, where, who, how many.* VFM answers *what happened, what did it look like, around when.* The router prefers the vault and falls back to the graph; neither replaces the other.

Part I
The graph memory (original architecture)

4 Four modules, four jobs

Module	Job
Perception	Motion gating, open-vocabulary detection, hand landmarks, tracking and Re-ID.
Memory	9-state lifecycle, support/containment reasoning, durable dual graph.
Semantics	Temporal strips, Florence-2 action verification, confidence calibration, GraphRAG.
Interface	CLI, operator dashboard, mission-control dashboard, uploads, export/import, query console.

The design principle is event-driven: a frame is not sent to a vision-language model unless geometry, motion or a state transition says something interesting happened. This is what makes the system fast enough to feel live and small enough to keep a whole episode on disk.

5 Data model: two graphs, one episode

Graph	What it stores
Background	Invariant anchors: zones (Kitchen, Work Desk) and fixed furniture (countertop, table, sofa). Keyframes are matched by CLIP place recognition at cosine ≥ 0.85.
Foreground	Episodic objects (keys, phone, mug) with persistent IDs, states, bounding boxes and time-intervaled relations.

Edges. Spatial (`RESTING_ON`, `INSIDE`, `IN_ZONE`), temporal (state transitions with start/end timestamps), and action multi-edges (`USER -[PICKED_UP @ t]-> KEYS`). The *terminal relational anchor* is never deleted: a long-lost object can still answer "where was it last?".

5.1 The 9-state lifecycle

State	Deterministic trigger (wall-clock, not frame count)
`NEW`	Detection with no track match and Re-ID cosine < 0.82.
`VISIBLE_STATIC`	Relative velocity ≈ 0 for > 0.5 s and support edge verified.
`INTERACTED`	Hand landmark containment or IoS(hand, object) > 0.60 for ≥ 0.3 s.
`VISIBLE_MOVING`	Object moves independently, hand IoS < 0.10.
`HELD`	Track persists, IoS > 0.50, object centroid follows hand velocity.
`CONTAINED`	Object collapses inside a container hull, IoS > 0.85.
`OCCLUDED`	Track lost while the camera is still and the last box overlaps an occluder.
`OUT_OF_VIEW`	Track lost during a camera pan; spatial anchor retained.
`LOST`	Out of view for > 600 s; node goes dormant, terminal anchor preserved.

All triggers are wall-clock or geometric, never frame counts. That keeps behaviour stable when the frame rate changes and makes the state machine testable without a model.

Key idea. The graph does not grow with frames. It grows with *identity and interaction.* Trajectories are bounded (10-minute purge), transitions and terminal anchors are retained. This is the storage argument in one sentence.

6 Perception: spend compute only where it matters

1. **Motion gate.** Mask dynamic boxes and hands, estimate a partial affine transform between frames, and classify the camera as stationary or panning. Below 2.5 px the scene is still; above 8 px it is a pan. This separates head rotation from real object motion.
2. **Detection.** YOLO-World is open-vocabulary but *prompt-driven*: it only labels the objects it is asked about. The starter list covers 12 objects; an opt-in broad preset covers 98 common indoor/egocentric objects.
3. **Hands.** MediaPipe returns 21 landmarks. Contact is a geometric test, not a guess.

4. **Tracking and Re-ID.** ByteTrack supplies kinematic IDs; inactive tracks are re-bound by CLIP appearance similarity (≥ 0.82). Graph-level consolidation merges legacy duplicates without merging co-visible objects.
5. **Interaction test.** Intersection over Smaller Area:

$$\mathrm{IoS}(A, B) = \frac{\mathrm{Area}(A \cap B)}{\min(\mathrm{Area}(A), \mathrm{Area}(B))}.$$

   Interaction fires when a fingertip enters the object hull, or IoS(hand, object) > 0.60 for ≥ 0.3 s.
6. **Temporal strip.** Keep a 30-frame RAM ring buffer. On an interaction, stitch three moments (pre / contact / post) into one 1×3 image and ask Florence-2 for an action caption. Strict JSON parsing and per-action thresholds reject hedged or implausible captions.

Constant	Value
Still / pan threshold	< 2.5 px / > 8 px affine magnitude
Re-ID cosine	0.82
Place-recognition cosine	0.85
Interaction / containment IoS	0.60 / 0.85
Interaction dwell	0.30 s
Temporal buffer	30 frames; strip panels 336 px
Lost / trajectory retention	600 s / 600 s
Verification queue	depth 8; dedup window 1024 clips
Language model	`qwen2.5:7b-instruct`; 4096 ctx; 30 min keep-alive

6.1 Object appearance attributes: *which* cup?

Every Nth detection crop gets a cheap, deterministic appearance pass on CPU: no model download, a few milliseconds per box. It answers the question a label alone cannot.

- **Dominant colour.** Trim 10% from every crop edge, resize the inner region to ≤ 64 px, run 3-means in BGR. Cluster weights are a Gaussian centre prior multiplied by a background-suppression term $\mathrm{clip}(\|c-b\|/80,\ 0.15,\ 1)$, where b is the median border colour. The central cluster wins when it covers $\geq 12\%$ of the crop. The primary BGR centroid becomes a stable colour word plus a `#rrggbb` swatch.
- **Colour word.** Achromatic pixels first ($V < 42$: black; $S < 35$: white/grey), then hue bands map to red, orange, yellow, green, cyan, blue, purple or pink.
- **Shape.** Bounding-box aspect ratio: ≤ 0.82 tall, ≥ 1.22 wide, otherwise square.
- **Size.** Relative crop area: $\leq 1.2\%$ small, $\leq 7.5\%$ medium, else large.
- **Brightness / saturation.** $V < 70$ dark, $V < 145$ dim, else bright; $S < 55$ muted, $S < 150$ colourful, else vivid.
- **Texture / position.** Laplacian variance ≥ 45 is detailed, else plain; the box centre gives left/centre/right and top/middle/bottom.
- **Stability.** A new colour replaces the track colour only when its confidence is at least 0.12 higher; geometry and position update immediately.

Attributes are persisted on the foreground entity and exposed in snapshot/CSV exports, graph tooltips and GraphRAG. Questions such as “what colour is the mug?”, “how big is it?” and “describe the backpack” cite the entity’s last observation with relation `APPEARANCE`; a missing attribute is answered as ungrounded rather than guessed. The offline evaluation set covers colour, size, describe and missing-detail cases and passes 13/13 with 100% citation precision.

7 Storage: why the graph is small

SQLite is a durable projection of the in-memory NetworkX graph, not a second copy. WAL mode, `synchronous=NORMAL`, foreign keys and indexes keep reads non-blocking: `(src, relation)`, `(dst, relation)`, `(t_start)`, `(entity_id, t_event)`.

- **Embeddings are per identity, not per frame.** A 512-d CLIP vector is stored once per entity/zone as float32 (2 KB), not for every observation.

- **Observations are trajectories only.** Bounding boxes and state, no pixels. Rows older than 10 minutes are purged. Anchors are never deleted.
- **Zones, furniture, transitions, edges, anchors and observations** are separate indexed tables; the schema is versioned in `meta`.
- **Export is portable.** `snapshot.json` contains graph, entities, actions, transitions, furniture, verification telemetry and backends; CSVs use stable headers.

Measured on this repo (45 s Aria kitchen clip)	Value
Source video	2.99 MB
Graph database, including WAL	0.436 MB
Entities / edges / transitions / anchors	11 / 53 / 214 / 28
Bounding-box observations	536 rows
Entity embeddings	22 KB
Naive per-observation 512-d embeddings	1.05 MB
Deterministic query latency	0.14–1.05 ms
Exported snapshot	190 KB

Key idea. Storage is $O(\text{identities} + \text{interactions})$, not $O(\text{frames})$. Trajectories are bounded by the 10-minute window; transitions and terminal anchors are retained. On very long videos transitions still grow, so retention is a policy decision, not a free lunch.

8 Query: graph first, language second

1. **Resolve** the entity (aliases, plurals, ambiguity-aware scoring) and the active zone. The result carries `match_score`, `confidence`, `match_reason` and ranked `candidates`.
2. **Serialize** the 2-hop subgraph: latest state, spatial anchor, last interaction, parent zone and the relevant action edges.
3. **Synthesize** either a deterministic template (always available) or prose from a local Qwen model through Ollama. The model sees only the table, never the video.
4. **Validate** every citation. A timestamp that does not map back to the subgraph is a hallucination and is dropped; if no valid citation remains, the answer falls back to the template.

Example. *"Where are my keys?"* → `The keys are resting on the dining table in the kitchen [t=16.4s].` The citation is clickable in the dashboard and seeks the video to 16.4 s. The newest grounded citation can also attach a supporting still frame (interaction strip, crop, zone keyframe, or an extracted video frame).

Why a small model is enough. The hard work is retrieval and grounding. The LLM's job is phrasing. That is also why the system survives with no model at all: the deterministic answerer uses the same subgraph.

Part II

The video-free memory (VLM layer)

9 Why a second layer?

The graph dashboard replays a live graph: entities, state transitions, spatial edges and zones. That is excellent for visual inspection, but it has two limitations for question answering:

- It only knows what the graph recorded. Replay clips often have **state/edge events but no VLM interaction table**, so "what happened around 10 s?" had no semantic answer.
- Labels can arrive late. If an entity row is emitted at `last_seen`, the first half of the episode shows anonymous `Track` rows.

VFM adds the layer the graph does not keep: **dense natural-language captions** at 30 s resolution, a compact event ledger derived from the graph, evidence snapshots, and a retrieval engine whose answers are deterministic. Three consequences matter:

1. **Semantics.** The graph knows `person VISIBLE_MOVING`; the caption knows "a person walks on a sidewalk holding a cup of coffee". The second is what a user asks for.
2. **Coverage.** Captions cover the whole episode, including segments where the tracker emitted no interaction row.
3. **Recall by time.** Caption windows are timestamped, so "around 10 s" is a range query, not a similarity search over one frame.

Key idea. The graph is the *index*; the captions and events are the *memory*. The graph records state and identity, VFM records appearance and activity, and the vault joins them.

10 Compile pipeline: pay once

1. **Snapshot the source DB.** SQLite's backup API makes a consistent copy while the ingest worker may still be writing.
2. **Caption once.** Split the video into 30 s windows (20 s for short clips); sample three frames per window; caption each frame with a VLM (`qwen2.5vl:3b` by default) and synthesise a window caption. Store zlib-compressed JSON with objects, actions, zone and tags.
3. **Build the event ledger.** Read entities, state transitions, anchors and edges; coalesce visibility noise and repeated touches into salient events. Pack them struct-of-arrays at roughly 17 bytes/event.
4. **Build the vault.** Write 1 KB SQLite pages, the string dictionary, packed events, captions and metadata (including per-label count summaries).
5. **Extract evidence.** Save a bounded set of 480 px WebP q75 stills (interaction strips, object crops, zone keyframes). No full-resolution frames are retained.
6. **Render the report.** Write a single-file HTML with the timeline, captions and a client-side query box; embed evidence as data URLs.
7. **Write the pack.** Produce an LLM-ready Markdown/JSON pack. Budgets of 2k/4k/8k tokens keep the verified answer and rules, then add windows/events until the ceiling.
8. **Zip and manifest.** Expose `vault.db`, `memory.html`, `ledger.json`, `memory_pack.md/json`, CSVs and `evidence/`.

The compile is the only expensive step (seconds to tens of seconds, dominated by VLM calls). In the UI it runs *once per clip, in the background*, on first view. After that, opening the clip or asking a question does not touch the video.

11 What is saved, and why it is more information

The graph is still saved. VFM adds four new artefacts. All are derived once, compressed, and stored next to the source run; the video itself is optional at query time.

Artefact	What it contains	Demo	Walking
Graph DB	entities, states, edges, zones, observations (source of truth)	168 KB	254 KB
Packed vault	struct-of-arrays events, string dictionary, metadata, answer cache	28.7 KB	31.7 KB
Caption ledger	30 s VLM windows: caption, objects, actions, zone, tags, 3 frame captions	1 window	2 windows
Event ledger	salience-coalesced state/edge/interaction events	8 events	11 events
Evidence	480 px WebP stills, ranked by interaction / crop / zone keyframe	8 frames	8 frames
Memory pack	verified answer + ranked captions/events + evidence refs + rules	2.4 KB	3.0 KB
Report	single-file offline HTML with timeline, captions, query box	366 KB	410 KB
Whole job dir	vault + ledger + evidence + report + pack + zip	**988 KB**	**1.3 MB**

The report and zip duplicate bytes for convenience; the *at-rest memory* is the vault + ledger + evidence. On a 60-minute synthetic episode (2,474 events, 184 caption windows, 40 snapshots) the vault is 142 KB and the whole memory stays far below the source video.

Why this is more information than the graph alone: semantics (a caption describes the scene, not just a state); coverage (the whole episode, not only interaction rows); recall by time (timestamped ranges); evidence (stored stills, seekable); counts (track / peak / physical estimates).

Key idea. The VFM artefacts are a lossy summary, deliberately. The design goal is not to preserve every pixel; it is to preserve enough *timestamped meaning* to answer repeated questions, while keeping a pointer back to the evidence.

12 Query path, latency and counts

12.1 Deterministic vault (the default)

The query engine resolves an intent — where / when / around / before / after / count / holding / inventory / yes-no / keyword — looks up entities and events by time, and returns an answer with citations and an evidence reference. A canonical answer cache makes paraphrases hit the same entry. No LLM, no video, no frame extraction in this path.

12.2 Optional Qwen (phrasing only)

The pack can be passed to a local text model for phrasing (`qwen2.5:7b-instruct`). The model never sees frames; it sees the verified structured answer, timestamped captions, events and evidence references. It is a polish layer, not a dependency.

Path (30-question internal suite)	Accuracy	p50	Video at answer time
Deterministic VFM vault	**30/30**	**0.05–0.06 ms**	no
Qwen 7B from memory pack	**30/30**	1.07 s	no
Original graph + Qwen 7B	14/30	0.85 s	yes
9-frame contact sheet + Qwen2.5-VL	7/30	4.46 s	yes (19.24 MB)

The 30-question set is an internal regression suite, not a public benchmark. It is useful because it covers location, temporal, count, negation, before/after and paraphrase questions on the same episodes.

12.3 Counts: tracks, visibility and duplicates

Count questions need more than one number. At compile time the system computes, per label:

Field	Meaning
`tracks`	raw number of tracker ids with that label (upper bound)
`peak`	max same-label tracks present in one observation frame (lower bound)
`physical`	tracks after removing cross-label duplicate detections and one-frame transients
`duplicates`	tracks that were near-identical to a track under another label
`merged_into`	which label absorbed those duplicates

A duplicate is accepted only when two different-label boxes share the same frame with IoU ≥ 0.90 and centre distance ≤ 5 px, and one label has more total observations. This is deliberately conservative: same-label co-visible boxes on cook have max IoU ≈ 0.01, so they are *not* merged.

On the cook clip this turns four “bowl” tracks into **zero physical bowls**: two of the plates were emitted under both `bowl` and `plate` (IoU 0.91–0.99), and a fourth track was a one-frame detection. The plates resolve to two physical plates. On walking, two car tracks are one object at a time, so the answer is a range (1–2), not a false single count.

12.4 Event-driven evidence and verification

The trigger for “something important happened” is the **graph state machine**, not the captioner. It emits transitions such as `NEW`, `INTERACTED`, `HELD`, `CONTAINED`, `PLACED_ON`, `LOST`. Those transitions drive two bounded systems:

- **Verification queue.** Interaction clips are de-duplicated, bounded and queued. A temporal strip gives the verifier pre/contact/post frames; the Florence-2 path returns an action primitive, confidence and a rejection reason. Rejected or ambiguous actions are inspectable in the review queue/telemetry.

- **Evidence selection.** At compile time, `extract_evidence.select_events` picks major markers (`INTERACTED`, `HELD`, `CONTAINED`, `LOST`, `NEW`, `PICK`, `PLACE`, `INSERT`, anchors/keyframes), enforces a 2 s minimum gap and a 40-still cap, then writes 480 px WebP stills with an append-only index.

The event-clip recorder implements the bounded version of that idea: state transitions, interactions, low-confidence/rejected actions and label conflicts get a readable salience score, coalesce within 2.5 s, and become a 3 s pre + 3 s post `clip.mp4`, a rebased `events.jsonl` replay and a `graph.json` snapshot. The zine toolbar exposes it as **events** (`POST /api/event-clips`); see `docs/EVENT_CLIPS.md`. There is still **no learned live anomaly score** and no VLM call on every strange moment: the VLM pass over selected clips and the evidence/review indexing remain future work. The demo therefore records *stills and bounded clips at meaningful transitions* and verifies *actions*, not every strange moment.

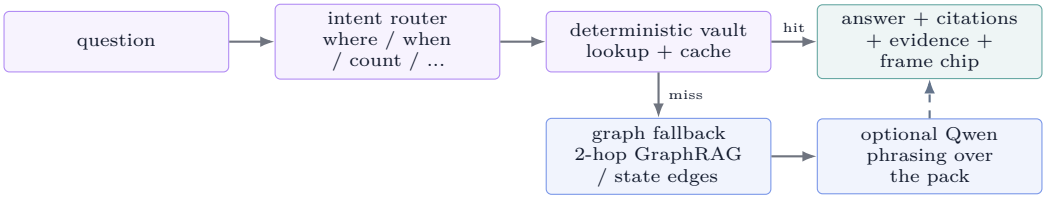
Part III
Putting it together

13 Two ways in: graph-backed vs caption-first

	A. Graph-backed clip	B. Caption-first clip
Prerequisite	A perception DB (bundled or ingested)	Just a video file
Step 1	Ingest: entities, states, edges, zones, observations	Upload / select the video
Step 2	Verify important actions (temporal strip + Florence-2)	Caption once: 30 s windows, 3 frames
Step 3	Compile: snapshot DB, caption, ledger, vault, evidence, pack, report	Build the caption vault, evidence and pack
Step 4	Answer from the vault; graph fallback on a miss	Answer from captions/events only
Counts	track / peak / physical reconciliation	caption coverage, clearly labelled as such
Inspect	citations, frame chip, evidence, export ZIP	citations, frame chip, evidence, export ZIP

The second path is what makes long street walks work without a perception pipeline. The trade-off is explicit: no graph means no state/edge fallback and no physical-count claim.

14 Routing a question



Question	First engine	Fallback / note
“Where are my keys?”	vault (where intent)	graph 2-hop: latest state + anchor
“What happened around 10 s?”	vault (around intent, caption windows)	graph “around t” state/edge lookup
“How many bowls?”	vault count reconciliation	graph track/peak/physical estimates
“What colour is the mug?”	vault caption or appearance relation	graph `APPEARANCE` attributes
“Describe the kitchen.”	vault caption windows	graph zone + keyframes
“Why did she pick it up?”	not reliably answerable	Qwen phrasing can only restate evidence
“What did the tiny object do?”	weakest case	evidence / frames on demand, honest limits

The router is deliberately simple: resolve intent and time, try the vault, fall back to the graph, and never claim more than the evidence supports.

15 Trace one question end to end

Question: "What happened around 10 seconds?"

Step	What happens	What the user sees
1	Parse intent `around` and time $t = 10\,\text{s}$	—
2	Open the vault; find caption windows and coalesced events covering the range	—
3	Rank the nearest window/events; attach citation timestamps	—
4	Compose the answer deterministically; cache the canonical form	prose + timestamp chips
5	Attach evidence reference if a still is within 8 s of the question	thumbnail
6	Render the `frame @ t` chip; capture/seeking happens in the UI, not the query	frame chip
7	If the vault has no match, route to the graph and label the answer `graph-fallback`	answer labelled with its source

Question: "Where are my keys?" takes a different first path in spirit: resolve the `keys` entity, read the latest state and spatial anchor from the 2-hop subgraph, validate the timestamp, then answer. In the combined system the vault usually hits first (a caption/event entry); the graph is the exact, structured fallback and the count authority.

Key idea. A good answer has three parts: a claim, a timestamped citation, and an evidence pointer. If any part is missing, the system should say so rather than guess.

16 Same UI, two engines

The zine mission control is the default UI. VFM is added behind it, not as a second uploader.

- **Auto-compile.** On first view of a clip with a graph DB, the vault builds in the background. The Memory card shows progress, then report and downloads.
- **VLM-first chat.** Once ready, the existing chat routes to the vault (`provider: vfm-memory`) and answers in microseconds. If not ready, a state/edge fallback still answers "around t" instead of claiming no interactions exist.
- **Screenshot at answer time.** Every grounded answer gets a frame chip that seeks to the citation, captures the exact frame, stamps clip + timestamp, and opens a lightbox.

- **Fullscreen.** Stage toolbar `S` (screenshot) and `F` (fullscreen); the screenshot lightbox has its own fullscreen button.
- **Downloads.** `vault.db`, `memory.html`, `memory_pack.md`, `memory_pack.json`, `memory.zip`.

Use the graph when...	**Use VFM when...**
You need exact state, support, containment or zone membership	You need "what happened", appearance or a description
You need a defensible physical count with duplicates resolved	You need an approximate count or coverage statement from captions
The clip predates captions or has no interaction table	The clip has been compiled (auto-compile)
You are debugging identity, tracking or the lifecycle	You are answering repeated questions in front of an audience

Part IV
Study aids

17 Numbers to remember

Area	**Value**	**Where it applies**
Motion gate	still < 2.5 px, pan > 8 px	perception
Re-ID / place cosine	0.82 / 0.85	tracking, zones
Interaction / containment IoS	0.60 / 0.85	state triggers
Interaction dwell	0.30 s	state trigger
Temporal ring buffer	30 frames, 1×3 strip	action verification
Purge / lost / anchor retention	600 s / 600 s / forever	storage
Caption window	30 s (20 s short clips), 3 frames	VFM compile
Event packing	≈ 17 bytes/event	vault
Evidence still	480 px WebP q75, 2 s gap, 40 cap	evidence
Pack budgets	2k / 4k / 8k tokens	memory pack
Count dedup	IoU ≥ 0.90, centre ≤ 5 px	counts
Graph query latency	0.14–1.05 ms	original engine
Vault query latency	0.05–0.06 ms demo; 0.89 ms cold, 3 ms cache	VFM engine
Qwen over pack	1.07 s (30/30)	optional phrasing
Graph DB size	0.436 MB for a 45 s clip	storage
Vault size	28.7 KB demo / 31.7 KB walk / 95 KB 15-min	storage

18 How this compares with common industry approaches

Approach	**Answer path**	**Strength**	**Weakness**
Frame-sampling video LLM	Sample N frames per question, send to a VLM	Open-ended reasoning, no compile step	Cost/latency scale with length; coverage depends on sampling; video must be available
Long-video RAG / frame retrieval	Embed frames/clips, retrieve top-k, then VLM	Better coverage than fixed sampling	Retrieval misses; VLM per query; frames remain the substrate
Hierarchical memory / caption QA	Compile captions/events once, retrieve text	Cheap reuse, long-horizon coverage, no video at query	Lossy captions; segmentation design; less fine-grained than pixels
Chronicle (combined)	Compile graph + captions + evidence; deterministic lookup; optional text LLM	**Sub-ms to ms; 249 MB → 95 KB; evidence-backed; offline; graph fallback**	Caption loss; internal benchmark; no audio; not general video reasoning

The honest claim is **complementary, not categorically better**. Industry systems optimise open-world reasoning; Chronicle optimises repeated, evidence-backed questions over a long episode.

19 Strengths, weaknesses and honest limits

Strengths	Weaknesses / limits
Speed and cost: answers from a 95 KB vault, not a 249 MB video; cold p50 sub-ms to a few ms, repeats cached	Lossy compression: captions summarise; fine motion, small text, audio and subtle social cues are lost
Coverage: captions cover the whole episode, including segments with no interaction row	Caption hallucination: the VLM can misdescribe an object; evidence makes it auditable, not infallible
Video-free: after compile, query/report/pack/evidence do not need the video	Counts: track/peak/physical are estimates, not hand-labelled truth for every episode
Evidence and audit: timestamped citations, stored stills, frame chips, fullscreen, ZIP export	Long video without a DB: no state/edge fallback and counts are caption coverage only
Deterministic: no model in the default answer path, so reproducible and cheap on stage	Internal evaluation: the 30-question suite is a regression test, not a public benchmark
Composable: graph is still identity/state/count authority and fallback	Weak on “why”, exact micro-motion, tiny objects and audio

Verdict. It is a *good retrieval-and-recall system*, not a replacement for a full video-reasoning model. It is strongest on “what happened / where / when / how many / what is X holding”, and weakest on “why / exactly how / what did the tiny object do”.

20 Demo script and pitfalls

The pitch in one line

“We turn a 15-minute, 249 MB video into a 95 KB searchable memory; questions are answered in under a millisecond, with no video and with evidence.”

30-second version

1. Everyone else re-watches or re-samples the video for every question. That is slow and expensive.
2. We compile once: VLM captions, a salience event ledger, a few evidence stills and a small memory pack.
3. After that, the vault answers deterministically in under a millisecond; an LLM is optional for phrasing only.
4. The same UI keeps the graph, playhead and evidence, and exports/imports the whole memory as a ZIP.

Ten-minute demo script

1. Start on the synthetic fixture. Show the graph, timeline and live pipeline log.
2. Ask *“Where are my keys?”* Click the citation: the video seeks to the moment.
3. Scrub back before the placement and ask again. The answer changes. That is episodic memory, not a transcript.
4. Switch to the HoloLens or Aria clip. Ask a multi-hop question: *“Where were the keys before the drawer?”*
5. Show the verification queue, calibration numbers and citation precision.
6. Stop Ollama or run `-no-llm`; repeat a query. The deterministic answer still cites the graph.
7. Play the 15-minute walk; show video size vs the 95 KB vault.
8. Ask “what happened at 10 minutes?” and “what is the person holding?”; show citations and the frame chip.
9. Ask a count question; show track/peak/physical and the deduped cook-bowls example.
10. Screenshot the cited frame, open fullscreen, export the graph ZIP, import it back.

Pitfalls to rehearse

1. YOLO-World only sees prompted labels — use *Broad scene* or name the objects.
2. Preload Qwen before going on stage; cold 7B load is dead air.
3. Checkpoint WAL before quoting database size: `PRAGMA wal_checkpoint(TRUNCATE)`.
4. Verify evidence images exist for real clips; otherwise the frame falls back to video extraction.
5. Do not claim Florence-2/OpenCLIP unless those extras are installed on the demo machine.
6. Do not present 30/30 as an external benchmark.

21 Self-check: 15 questions

Answer these without looking. If you can do 12, you understand the system.

1. **What are the two memory substrates?** *The per-episode graph DB (identity, state, edges, zones) and the VFM vault (packed events, captions, metadata, cache).*
2. **Why is the graph small?** *It grows with identities and interactions, not frames; trajectories are purged after 10 minutes and embeddings are stored once per identity.*
3. **What triggers `INTERACTED`?** *A fingertip inside the object hull, or IoS(hand, object)* > 0.60 *for at least 0.3 s.*
4. **What is a terminal anchor?** *The last verified spatial relation of an object; it is never deleted, so "where was it last?" always has an answer.*
5. **Why does VFM exist?** *The graph often lacks a semantic interaction table; VFM adds timestamped captions, an event ledger and deterministic recall with no video at query time.*
6. **What is salience coalescing?** *Filtering visibility noise (pan, occlusion, reacquisition) and merging repeated touches/placements into memory-worthy events.*
7. **What does the vault contain?** *Packed struct-of-arrays events, a string dictionary, captions, per-label count summaries, metadata and an answer cache.*
8. **Why is Qwen optional?** *It only phrases the verified structured answer; the deterministic path is complete without it.*
9. **What is the difference between tracks, peak and physical?** *Tracks = raw ids (upper bound); peak = most same-label boxes in one frame (lower bound); physical = tracks after duplicate/transient removal (estimate).*
10. **When does a duplicate count as merged?** *Different-label boxes in the same frame with* $IoU \geq 0.90$*, centre distance* ≤ 5 *px, and one label having more observations.*
11. **What does the frame chip do?** *Seeks to the cited time, captures that frame, stamps clip + timestamp, and opens it in a save/fullscreen lightbox.*
12. **What happens when the vault has no answer?** *The same chat asks the live graph engine and labels the answer* `graph-fallback`*.*
13. **What is lost by compiling?** *Fine motion, audio, tiny text, subtle social cues and some rare objects; captions are a summary and can hallucinate.*
14. **Is 30/30 a benchmark?** *No — it is an internal 30-question regression suite covering location, temporal, count, negation, before/after and paraphrase cases.*
15. **What is the one-sentence pitch?** *Compile a long video into a tiny timestamped memory once, then answer evidence-backed questions in under a millisecond without the video.*

22 Glossary

Appearance relation
Persisted colour/shape/size/texture/position attributes attached to an entity with relation `APPEARANCE`.

Auto-compile
Building the vault once, in the background, on first view of a clip. Queries stay on the graph fallback until it is ready.

Caption ledger
zlib-compressed JSON of 30 s (or 20 s) windows. Each window has a caption, objects, actions, zone, tags and three timestamped frame captions.

Deterministic query
Intent + time + entity lookup over the vault. No LLM, no video, no frame sampling; answers carry citations.

Evidence frame
The still that supports an answer: interaction strip, object crop, zone keyframe, or extracted video frame.

Event ledger
Salience-coalesced state/edge/interaction events from the graph, packed at about 17 bytes per event.

Frame chip
The per-answer `frame @ t` control: seek, capture, save/fullscreen.

Graph fallback
If VFM has no answer, the same chat asks the live graph engine and labels the answer `graph-fallback`.

GraphRAG
Retrieve a 2-hop subgraph, then synthesize a grounded answer.

IoS
Intersection over Smaller Area; containment-friendly alternative to IoU.

Memory pack
LLM-ready Markdown/JSON: verified answer + ranked captions/events + evidence references + answering rules. Budgeted at 2k/4k/8k tokens.

Peak
Most same-label tracks visible in one observation frame (lower bound on count).

Physical count
Tracks after cross-label duplicate and one-frame transient removal (estimate).

Re-ID
Re-identification; CLIP appearance matching that re-binds a lost track to an entity.

Salience coalescing
Filtering visibility noise and merging repeated touches/placements into memory-worthy events.

Terminal anchor
The last verified spatial relation of an object; never deleted.

Track
Raw tracker id count for a label (upper bound on count).

Vault
The compact SQLite file written by the compiler: struct-of-arrays events, string dictionary, captions, evidence index, metadata and answer cache.

VFM
Video-Free Memory: the compiled artefact that answers questions without the source video.

VLM
Vision-language model; used once at compile time to caption frames/windows (default `qwen2.5vl:3b`), not in the deterministic answer path.

VPR
Visual place recognition; CLIP keyframe matching for zone relocalization.

23 Commands and file map

```
# Start the merged zine UI (graph UI + VFM memory)
cd /home/g1/projects/chronicle-vfm
./chronicle.sh start            # this worktree defaults to :8082
./chronicle.sh status
./chronicle.sh stop

# Full verification
./chronicle.sh test

# Ingest / query from the CLI
chronicle sources
chronicle run --source synthetic
chronicle ask "where are my keys?" --db data/demo/chronicle.db

# Compile a memory directly (no UI)
python prototype/compile.py --source-db data/real/cook/chronicle.db --out /tmp/cook.db
python prototype/attach_ledger.py --vault /tmp/cook.db --ledger /tmp/cook_ledger_30.json
python prototype/query.py --db /tmp/cook.db "how many bowls?"
python prototype/report.py --db /tmp/cook.db --out /tmp/cook.html --data-root /tmp
python prototype/memory_pack.py --db /tmp/cook.db --list-budgets
```

Path	Role
`chronicle/perception/`	Motion gate, detector, hands, tracker, interaction geometry.
`chronicle/graph/`	Foreground/background graphs, state machine, query engine, consolidation.
`chronicle/storage/`	SQLite schema and durable projections.
`chronicle/semantic/`	VPR, temporal strips, Florence-2, actions, queue, GraphRAG, LLM.
`chronicle/integration.py`	Runtime facade used by CLI, dashboards and demos.
`dashboard/`	Mission-control dashboard, query console, API, evidence frames.
`chronicle/ui/`	Operator dashboard, uploads, snapshot import/export.
`prototype/`	VFM compiler, ledger, query, report, pack and evidence extractor.
`docs/`	Contracts, role notes, evaluation examples, this guide.

Configuration cheat sheet

Variable	Default / meaning
`CHRONICLE_LLM_BACKEND`	`auto`, `ollama`, `transformers`, `template`, `none`
`CHRONICLE_LLM_MODEL`	`qwen2.5:7b-instruct`
`CHRONICLE_OLLAMA_URL`	`http://127.0.0.1:11434`
`CHRONICLE_QUERY_URL`	Optional external answer endpoint
`CHRONICLE_DETECTOR`	`auto`, `color`, `yolo-world`
`CHRONICLE_HANDS`	`auto`, `color`, `mediapipe`
`CHRONICLE_EMBEDDER`	`auto`, `color`, `deterministic`, `openclip`
`CHRONICLE_SOURCE`	`demo`, `live`, `jsonl`, `sqlite`
`CHRONICLE_DATA_DIR`	Asset root (default `./data`)

24 Further reading

The design borrows one idea from each line of work. Citation keys point to the entries below.

Work	Idea borrowed	Link
CapMem	Captions-only episodic QA; 30 s caption ledger	arXiv:2609.17688
EgoCITE	Time as a first-class retrieval key	arXiv:2608.12627
MERIT	Query-agnostic, high-recall memory build	arXiv:2608.07663
R4DSG	Anchor-relative 4D scene memory	arXiv:2608.11017
VL-KnG	Persistent spatio-temporal KG; graph retrieval	arXiv:2510.01483
EgoGraph	Egocentric entity/event schema	arXiv:2602.23709
FocusGraph	Text-first, pixels-on-demand routing	arXiv:2603.04349
LightMem	Sensory → short-term → long-term tiers	arXiv:2510.18866
HippoRAG	Graph retrieval as a cheap alternative to iterative VLM calls	arXiv:2405.14831
VideoMem	Bounded retention / adaptive memory buffer	arXiv:2512.04540
EM-LLM	Event segmentation + temporal-contiguity retrieval	ICLR 2025
Caption-once, Frames-on-Demand	Cache captions once; fetch frames only when needed	EMNLP 2026
Titans / MemoryLLM	Model-native long-term memory (north star, not a dependency)	arXiv:2501.00663

How to keep learning. Rebuild the two source PDFs, change one constant, and re-run the 30-question suite; then add one new intent to the router and trace it through the vault and the graph fallback. The system is small enough that this loop is the fastest way to internalise it.

End of guide — graph memory + video-free memory, one system.